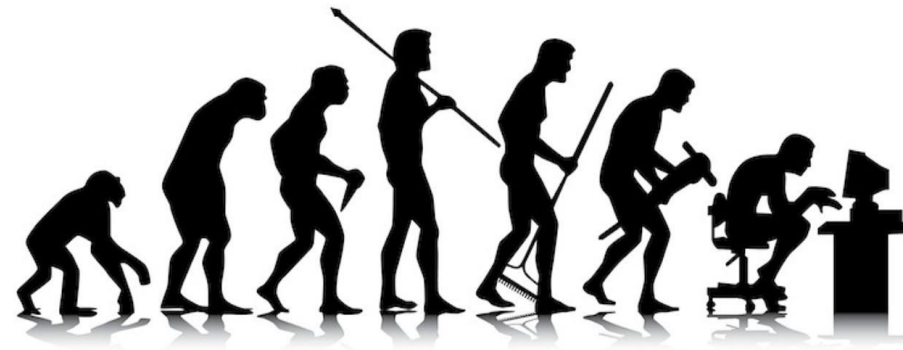


Collections Aufgaben



One-dimensional String-Theory

1. Erstelle ein Array "names" mit folgenden Strings:
"Chani", "Sara", "Leto"
2. Iteriere über das Array mit einem Iterator und gebe jeden Namen in GROßBUCHSTABEN in der Konsole aus.
3. Iteriere nochmal über das Array, dieses Mal mit einer for-Schleife. Ersetze hierbei in jeden Namen das "o" mit einem "i" und gebe den neuen Namen in der Konsole aus.

Kochbuch

1. Schreibe eine Klasse (oder Record) „Recipe“, die folgende Attribute enthält:
 - name
 - description
2. Schreibe eine main-Methode.
3. Erzeuge in der main-Methode eine Liste „recipes“ und füge einige ausgedachte Recipe-Objekte hinzu.
4. Gebe in der main-Methode alle enthaltenen Recipes in der Konsole aus.
5. Optional:
Erzeuge und überschreibe die toString()-Methode in der Klasse Recipe, um die Ausgabe zu manipulieren.

UniqueCharactersCounter

Schreibe eine Methode `countUniqueCharacters`, die die Anzahl der einzigartigen Zeichen in einem gegebenen String zurückgibt.

Verwende ein `HashSet`.

```
public static int countUniqueCharacters(String str)
```

Dictionary

1. Schreibe eine main-Methode.
2. Erstelle in der main-Methode eine Map, in der englische und deutsche Wörter zugeordnet sind. Denke dir hierzu einige Key-Value-Beispiele aus und füge sie deiner Map hinzu:
„hello“ -> „Hallo“
„Hallo“ -> „hello“
„cat“ -> „Katze“
„Katze“ -> „cat“
3. Lese einen String von der Konsole ein. Stichwort „[Scanner](#)“.
4. Gebe die Übersetzung des eingegebenen Wortes in der Konsole aus.
5. Gebe “sacre bleu” aus, falls keine Übersetzung vorhanden ist.



Object Caster

Schreibe eine Methode, die eine Object-Liste von Zahlen erhält.

Iteriere über die Liste und berechne für jede Zahl das Quadrat.

Gebe die Quadrate als neue Liste zurück.

```
public static List<Integer> toSquaredIntegerList(List<Object> objectList)
```

Falls ein Object nicht gecastet werden kann, soll das Objekt **nicht** in die Ergebnisliste mit aufgenommen werden.

Schreibe einen Unit Test die Klasse.

ListConverter

Schreibe eine Klasse ListConverter und implementiere die folgenden Methoden:

// Wandle die gegebene List<Integer> in ein Integer[]-Array um und gebe das Array zurück.

```
public Integer[] toArray(List<Integer> list)
```

// Wandle das gegebene Integer[]-Array in eine List<Integer> um und gebe die List zurück.

```
public List<Integer> toList(Integer[] array)
```

Schreibe einen Unit Test die Klasse.

MyStack<T>

Implementiere eine generische Klasse MyStack<T>, die **intern** eine **LinkedList** verwendet und folgende Methoden besitzt:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

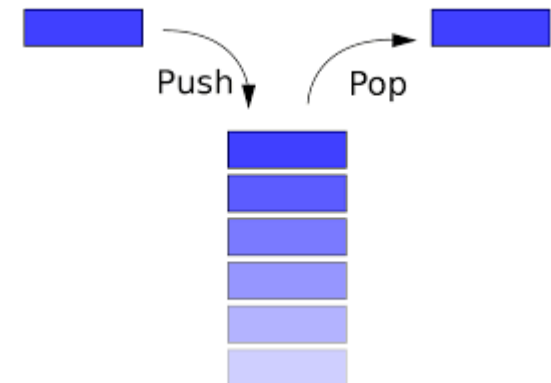
// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen.

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```

Schreibe einen Unit Test die Klasse.



Longest String

Finde in einer Collection den längsten String und gebe ihn zurück.
Falls kein String gefunden wird, gebe einen leeren String "" zurück.

Beispiel:

```
Collection<String> words = List.of("Apple", "Orange", "Strawberry", "Banana");  
String longestString = LongestString.findLongestString(words); // Strawberry
```

Überlege dir, welcher Edge-Case hier wahrscheinlich auftreten könnte.
Schreibe einen Unit Test für deine Lösung.

Linear Search

Implementiere eine Methode, die eine **Collection** von ganzen Zahlen sowie eine gesuchte Zahl entgegennimmt.

Die Methode gibt den **Index** der gesuchten Zahl in der **Collection** zurück, falls die Zahl vorhanden ist, sonst -1.

Bei mehrfachen Vorkommen der gesuchten Zahl, soll der **Index** der zuerst gefundenen Zahl zurückgegeben werden.

Tipp: Es könnte hilfreich sein, die empfangene Collection in eine Liste umzuwandeln.

Beispiel:

```
Collection<Integer> numbers = List.of(1, 2, 3, 4, 5);  
int index = LinearSearch.findIndexOfNumber(numbers, 3); // 2
```

Schreibe einen Unit Test für deine Lösung.

Find MinMax

Implementiere eine Methode, die eine Collection von Ganzzahlen entgegennimmt und ein **record** MinMax mit zwei Attributen zurückgibt: dem kleinsten und dem größten **Wert** in der Eingabe-Collection.

Falls nur ein Element vorhanden ist, ist dieses Element zugleich Minimum und Maximum.

Wenn keine gültigen Werte gefunden werden, wird eine **NoSuchElementException** geworfen.

Beispiel:

```
Collection<Integer> integers = List.of(1, 2, 3, 4, 5, 0);  
MinMax minMax = MinMaxUtil.findMinMax(integers); // MinMax[min: 0, max: 5]
```

Schreibe einen Unit Test für deine Lösung.